5. `agent.py` expliqué

Nutri-Arbitre — fiche de révision · étape 5 sur 5 du trajet des données (download → clean → tools → garde_fou → **agent**) · **brique 1** (appel d'outils)

En une phrase : ce fichier est le **chef d'orchestre**. Il envoie la question à Qwen, exécute les outils que Qwen demande, lui renvoie les résultats, recommence jusqu'à ce que Qwen donne sa réponse finale, puis fait passer cette réponse par le garde-fou avant de l'afficher.
Brique 1 (appel d'outils) : Qwen ne calcule rien lui-même. Il *demande* un outil (« decrire les sucres des bio »), et c'est le code Python de `tools.py` qui fait le calcul.
Qwen ne lance rien lui-même. Il écrit seulement une demande : le nom d'un outil et ses arguments. C'est `agent.py` qui décide de l'exécuter ou non.

Le trajet d'une question

```
question ──► Qwen ──► « appelle decrire(sucres, bio=True) »
                            │
               agent.py vérifie les filtres (garde-fou n° 2), puis exécute
tools.decrire
                            │
              résultat {"n": 543, "moyenne": 13.17, ...} ──► renvoyé à Qwen
                            │
               Qwen écrit sa réponse en français (plus aucun outil demandé)
                            │
               garde-fou n° 1 : chaque nombre vient-il d'un outil ? ──► réponse
affichée ou bloquée
```

Les réglages (lignes 8 à 23)

```
19 MODELE = os.environ.get("NUTRI_MODELE", "qwen3.5:4b-q4_K_M")
20 OPTIONS = {"num_ctx": 8192}
21 MAX_TOURS = 6
22 DONNEES = Path("data/cereales_propres.csv")
23 DOSSIER_TRACES = Path("traces")
```

- **MODELE** : `os.environ.get("NUTRI_MODELE", ...)` lit une variable d'environnement. Si elle n'existe pas, on prend la valeur par défaut, le modèle imposé par la consigne. On peut ainsi tester un autre modèle sans changer le code.
- **num_ctx = 8192** : la **longueur de contexte**, c'est-à-dire combien de tokens (morceaux de mots) le modèle peut lire en une fois. La consigne demande de la fixer. Toute la conversation (prompt, question, résultats d'outils) doit tenir dedans.
- **MAX_TOURS = 6** : une sécurité. Si Qwen demande des outils sans jamais conclure, on arrête au bout de 6 tours au lieu de tourner à l'infini.
- **Path** : un chemin de fichier. `DOSSIER_TRACES / "trace.json"` colle les morceaux du chemin.

Le prompt système (lignes 25 à 30)

C'est le texte qui explique à Qwen son rôle, envoyé avant chaque question. Il fixe :

- le **rôle** (analyste nutrition) et le **sujet** (céréales vendues en France, pour 100 g) ;
- la **règle** : tout chiffre vient d'un outil, et on recopie les nombres tels quels ;
- la **forme** : 2 à 4 phrases, la marge d'erreur écrite « 13,2 g ± 0,9 g » (brique 8) ;
- la **prudence** : si `difference_nette` vaut false, dire qu'on ne peut pas conclure.

> Le prompt **demande**, le garde-fou **impose**. Qwen ne respecte pas toujours le prompt (il oublie parfois la marge, ou calcule une portion lui-même). C'est pour ça que la règle des chiffres est aussi vérifiée par le code.

Charger les données (lignes 33 et 34)

```
34      return pd.read_csv(DONNEES, dtype={"code": str})
```

`dtype={"code": str}` : la colonne `code` (code-barres) est lue comme du **texte**. Sinon pandas la lirait comme un nombre et supprimerait les zéros du début (« 0012345 » deviendrait 12345).

`executer_outil` : exécuter la demande de Qwen (lignes 37 à 52)

```
40      if nom not in tools.FONCTIONS:
41          return {"erreur": f"outil inconnu : {nom}"}
43      probleme = garde_fou.verifier_filtres(question, arguments)
44      if probleme is not None:
45          return {"erreur": probleme}
46      fonction = tools.FONCTIONS[nom]
47      try:
50          return fonction(df, **arguments)
51      except (ValueError, TypeError) as e:
52          return {"erreur": str(e)}
```

Trois contrôles, dans l'ordre :

1. **L'outil existe-t-il ?** (lignes 40–41). `tools.FONCTIONS` est un dictionnaire nom → fonction (`"decrire"` → `tools.decrire`). Si Qwen invente un outil, on refuse. Seules nos 4 fonctions peuvent être lancées : Qwen ne peut pas exécuter n'importe quel code.
2. **Les filtres sont-ils permis ?** (lignes 43–45). C'est la deuxième règle du garde-fou : pas de filtre bio ou Nutri-Score si la question n'en parle pas.
3. **Les arguments sont-ils corrects ?** (lignes 47–52). Si Qwen demande un nutriment qui n'existe pas (« vitamine_c »), `tools.py` lève une `ValueError`. Un argument inconnu donne une `TypeError`. On **attrape** l'erreur avec `try / except`.

Le `**arguments`

Qwen envoie ses arguments sous forme de dictionnaire : `{"nutriment": "sucres", "bio": True}`. Les deux étoiles « déballent » le dictionnaire : la ligne 50 revient à écrire `tools.decrire(df, nutriment="sucres", bio=True)`. Ainsi, une seule ligne marche pour les 4 outils.

Pourquoi renvoyer l'erreur au lieu de planter ?

L'erreur est renvoyée **à Qwen**, comme un résultat d'outil. Il la lit au tour suivant et peut corriger son appel. Si le programme plantait, l'utilisateur n'aurait aucune réponse.

`repondre` : la boucle d'agent (lignes 55 à 105)

La conversation (lignes 58–59)

```
58      messages = [{"role": "system", "content": PROMPT_SYSTEME},
59                  {"role": "user", "content": question}]
```

Un modèle de langage n'a **pas de mémoire** d'un appel à l'autre. À chaque tour, on lui renvoie **toute** la conversation, sous forme d'une liste de messages. Chaque message a un **rôle** :

Rôle	Qui parle	Contenu
`system`	nous, une fois au début	le prompt système (les règles)
`user`	l'utilisateur	la question
`assistant`	Qwen	une demande d'outil, ou sa réponse finale
`tool`	nos outils Python	le résultat du calcul, en texte JSON

La trace (lignes 61–63)

`trace` est un dictionnaire qui garde tout ce qui se passe : la question, chaque appel d'outil avec ses arguments et son résultat, la réponse, les tokens, la durée. Il sera enregistré dans `traces/` : ce sont nos **preuves** (EVIDENCE.md, brique 9). `resultats` garde tous les résultats d'outils pour le garde-fou de la fin.

Avec ou sans outils (lignes 65–68)

Si `avec_outils` est faux, on ne propose aucun outil à Qwen : il doit répondre de mémoire. C'est le « modèle seul », qu'on compare au système complet dans la brique 9.

La boucle : 3 étapes par tour (lignes 70–93)

```
70      reponse_brute = None
71      for tour in range(MAX_TOURS):
73          r = chat(model=MODELE, messages=messages, tools=outils_proposes,
74                   options=OPTIONS, think=False)
75          trace["tokens_entree"] += r.prompt_eval_count or 0
76          trace["tokens_sortie"] += r.eval_count or 0
77          messages.append(r.message)
80          if not r.message.tool_calls:
81              reponse_brute = r.message.content or ""
82              break
85          for appel in r.message.tool_calls:
86              nom = appel.function.name
87              args = dict(appel.function.arguments or {})
88              resultat = executer_outil(df, nom, args, question)
89              resultats.append(resultat)
90              trace["appels"].append({"outil": nom, "arguments": args, "resultat":
resultat})
91              messages.append({"role": "tool", "tool_name": nom,
92                               "content": json.dumps(resultat, ensure_ascii=False)})
```

1. **On envoie la conversation à Qwen** (lignes 73–77) avec la liste des outils (`tools.SCHEMAS`, qui décrit chaque outil : son nom, à quoi il sert, ses arguments et leurs valeurs permises). `think=False` coupe la « réflexion » cachée de Qwen : plus rapide. On compte les tokens (`or 0` : si Ollama ne donne pas le nombre, on ajoute 0 au lieu de planter). La réponse de Qwen est ajoutée à la conversation.
2. **Pas d'outil demandé ? C'est la réponse finale** (lignes 80–82). `break` sort de la boucle.
3. **Sinon, on exécute chaque outil demandé** (lignes 85–92) et on ajoute le résultat à la conversation, avec le rôle `tool`. `json.dumps` transforme le dictionnaire en texte, car un message est toujours du texte. `or {}` : un outil sans argument (`compter`) peut arriver avec `None`. Puis la boucle recommence : Qwen lit les résultats et continue.

La fin (lignes 95–105)

```
95      if reponse_brute is None:
96          reponse_brute = "Je n'ai pas réussi à répondre en un nombre raisonnable
d'étapes."
99      if avec_outils:
100         trace["reponse"] = garde_fou.filtrer_reponse(reponse_brute, resultats,
question)
101     else:
102         trace["reponse"] = reponse_brute
103     trace["bloquee"] = trace["reponse"] != reponse_brute
```

- `reponse_brute` vaut encore `None` seulement si les 6 tours sont passés sans réponse finale : on met un message d'échec.
- **Le garde-fou n° 1** (ligne 100) s'applique au système complet. On garde les deux versions dans la trace : `reponse_brute` (ce que Qwen a écrit) et `reponse` (ce qui est affiché).

- `bloquee` vaut `True` si les deux sont différentes : le garde-fou a remplacé la réponse. On peut ainsi compter les réponses bloquées dans la brique 9.

> **Exemple réel, trace du 2 novembre (`trace_20261102_113921.json`)** : « Combien de calories apporte une portion de 30 g de céréales, en moyenne ? »
>
> | Tour | Ce qui se passe |
> |---|---|
> | 1 | Qwen demande `decrire(energie_kcal, bio=False)` . `executer_outil` refuse : « la question ne parle pas de bio… ». Ce refus est renvoyé à Qwen (rôle `tool`). |
> | 2 | Qwen corrige : `decrire(energie_kcal)` → n = 953, moyenne 409,45 kcal, marge 2,54. |
> | 3 | Pas d'outil demandé : réponse finale. Elle contient « 122,84 kcal (409,45 × 0,30) », un calcul fait par Qwen. |
> | fin | Le garde-fou bloque : 122,84 et 0,30 ne viennent pas d'un outil. `bloquee = True` . 3 497 tokens lus en tout, 17,8 s. |
>
> On voit la boucle en action : 3 appels à Qwen, une erreur qu'il corrige seul, puis la barrière finale.

Enregistrer la trace (lignes 108 à 112)

```
109     DOSSIER_TRACES.mkdir(exist_ok=True)
110     chemin = DOSSIER_TRACES / f"trace_{time.strftime('%Y%m%d_%H%M%S')}.json"
```

`mkdir(exist_ok=True)` crée le dossier `traces/` s'il n'existe pas (et ne fait rien s'il existe). Le nom du fichier contient la date et l'heure (`20261102_113921` = 2 novembre 2026, 11 h 39 min 21 s) : chaque trace a un nom différent, aucune n'en écrase une autre.

Le lancement dans le terminal (lignes 115 à 124)

```
python -m nutri.agent "Les céréales bio contiennent-elles moins de sucre ?"
python -m nutri.agent --modele-seul "Les céréales bio contiennent-elles moins de sucre
?"
```

- `if __name__ == "__main__":` : ce bloc ne tourne que si on lance le fichier directement, pas quand un autre fichier l'importe (les tests importent `executer_outil`).
- `argparse` lit ce qu'on tape après la commande : la question, et l'option `--modele-seul` . `action="store_true"` : l'option vaut `True` si on l'écrit, `False` sinon.
- `avec_outils=not args.modele_seul` : avec l'option, pas d'outils ni de garde-fou.
- On affiche chaque appel d'outil, la réponse, et le chemin de la trace.

Les limites (à dire soi-même)

- **Qwen ne répond pas deux fois pareil** : à la même question, il a choisi trois filtres différents en 10 minutes. C'est pour ça que la brique 9 lance chaque question au moins 3 fois.

- **Le choix de l'outil reste celui de Qwen** : s'il demande le mauvais nutriment, aucun contrôle ne le voit.
- **Le coût grandit à chaque tour** : toute la conversation est renvoyée, donc les tokens lus augmentent (environ 2 300 avec un outil, 3 500 avec deux tours d'outils).
- **Les questions de portion** sont toujours bloquées : aucun outil ne fait ce calcul.

Questions d'examinateur

> 1. Qui fait le calcul de la moyenne : Qwen ou Python ? Comment Qwen obtient-il le résultat ?
> 2. Pourquoi renvoie-t-on toute la liste `messages` à chaque tour ?
> 3. À quoi sert `MAX_TOURS` ? Que se passe-t-il si on l'atteint ?
> 4. Comment la boucle sait-elle que Qwen a fini ?
> 5. Expliquez `fonction(df, **arguments)` avec un exemple.
> 6. Pourquoi `executer_outil` renvoie-t-il `{"erreur": ...}` au lieu de laisser le programme planter ?
> 7. Qwen peut-il exécuter n'importe quel code sur votre ordinateur ?
> 8. Quelle est la différence entre `reponse_brute` et `reponse` dans la trace ?
> 9. À quoi sert l'option `--modele-seul` ?
> 10. Racontez une vraie trace où la boucle a fait plusieurs tours.

Corrigé

1. **Python.** Qwen écrit seulement une demande (nom d'outil + arguments). `agent.py` exécute la fonction de `tools.py` et ajoute le résultat à la conversation avec le rôle `tool` ; Qwen le lit au tour suivant.
2. **Le modèle n'a pas de mémoire** d'un appel à l'autre. Pour qu'il sache la question, ses demandes précédentes et les résultats, il faut tout lui renvoyer à chaque fois.
3. **Une sécurité contre une boucle sans fin** : au bout de 6 tours sans réponse finale, on s'arrête et on répond « Je n'ai pas réussi à répondre en un nombre raisonnable d'étapes ».
4. Quand la réponse de Qwen **ne contient aucune demande d'outil** (`tool_calls` vide) : c'est sa réponse finale, et `break` sort de la boucle.
5. Les deux étoiles déballent le dictionnaire en arguments nommés. Avec `{"nutriment": "sucres", "bio": True}`, la ligne revient à `tools.decrire(df, nutriment="sucres", bio=True)`. Une seule ligne sert pour les 4 outils.
6. Pour que **Qwen lise l'erreur et corrige son appel**. Le 2 novembre, il a demandé un filtre non bio, a reçu l'erreur, et a refait l'appel sur toutes les céréales.
7. **Non.** Il ne peut demander que les 4 outils de `tools.FONCTIONS`. Un autre nom est refusé (« outil inconnu »), et le test `test_outil_inconnu_ne_plante_pas` le vérifie.
8. `reponse_brute` est ce que Qwen a écrit ; `reponse` est ce qui est affiché après le garde-fou. Si elles diffèrent, la réponse a été bloquée (`bloquee = True`).
9. À lancer Qwen **sans outils ni garde-fou**, pour comparer avec le système complet dans la brique 9 : on mesure ce qu'apportent les outils et la règle.
10. La trace `113921` : 1er tour, filtre non bio refusé ; 2e tour, `decrire(energie_kcal)` sur les 953 céréales ; 3e tour, réponse avec « 122,84 kcal » calculé par Qwen ; le garde-fou la bloque.